

Tugas 2: Machine Learning – K-Nearest Neighbor

Muhammad Zaidan Ramdhan - 0110222040

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: muha22040ti@student.nurulfikri.ac.id

Laporan ini bertujuan untuk mengimplementasikan metode KNN dalam melakukan klasifikasi terhadap dataset yang diperoleh.

Pada Latihan1, proses klasifikasi menggunakan algoritma K-Nearest Neighbor (K-NN) berhasil menentukan persepsi suhu berdasarkan temperatur dan kecepatan angin. Hasil klasifikasi menunjukkan bahwa kondisi dengan suhu 16°C dan kecepatan angin 3 km/jam termasuk dalam kategori “Dingin”. Secara keseluruhan, model mampu memberikan prediksi yang sesuai dengan pola data yang tersedia.

Pada studi kasus latihan2, model machine learning telah menghasilkan prediksi kelulusan mahasiswa. Oleh karena itu, penelitian ini berfokus pada proses evaluasi performa model menggunakan confusion matrix beserta metrik klasifikasi seperti accuracy, precision, dan recall.

Pada studi kasus latihan2, model klasifikasi K-Nearest Neighbors (KNN) dengan K=5 telah dievaluasi untuk memprediksi kondisi cuaca (clear, overcast, partly cloudy) dan menghasilkan Akurasi keseluruhan 66.67%. Analisis metrik rinci per kelas menunjukkan kinerja yang sangat baik untuk memprediksi cuaca clear (*Precision* dan *Recall* 1.00). Namun, model menunjukkan kelemahan serius pada kategori partly cloudy, dengan nilai *Precision*, *Recall*, dan *F1-Score* 0.00.

1. Tugas mandiri – Latihan1

```
1. Melakukan mounting G-Drive

from google.colab import drive
drive.mount("/content/gdrive")

Mounted at /content/gdrive
```

Gambar 1. 1

Pada *Gambar 1.1* di atas, merupakan sebuah code untuk mounted atau menghubungkan google colab dengan google drive.

```
import math
from sklearn.neighbors import KNeighborsClassifier
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score, classification_report
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
```

Gambar 1. 2

Pada *Gambar 1.2* di atas, merupakan sebuah code untuk import beberapa library yang akan kita gunakan pada latihan1, latihan2 dan latihan3.

```
df = pd.read_excel(path + '/praktikum_mandiri/data/latihan1.xlsx')
df
```

	No	suhu	angin	kelas
0	1	10	0	dingin
1	2	25	0	panas
2	3	15	5	dingin
3	4	20	3	panas
4	5	18	7	dingin
5	6	20	10	dingin
6	7	22	5	panas
7	8	24	6	panas

Gambar 1. 3

Pada *Gambar 1.3* di atas, merupakan code untuk menampilkan dataset yang digunakan untuk latihan1. Pada sebuah tabel menggunakan dataset klasifikasi suhu.

```
def euclidean_distance(x1, x2):  
    return math.sqrt((x1[0]-x2[0])**2 + (x1[1]-x2[1])**2)
```

Gambar 1. 4

Pada *Gambar 1.4* di atas, merupakan sebuah code untuk menghitung jarak euclidean secara manual.

```
print("=== Hasil Perhitungan Jarak Manual ===")  
for i, (d, label) in enumerate(distances):  
    print(f"{i+1}. Jarak: {d:.3f} -> Kelas: {'Panas' if label else 'Dingin'}")  
  
=== Hasil Perhitungan Jarak Manual ===  
1. Jarak: 2.236 -> Kelas: Dingin  
2. Jarak: 4.000 -> Kelas: Panas  
3. Jarak: 4.472 -> Kelas: Dingin  
4. Jarak: 6.325 -> Kelas: Panas  
5. Jarak: 6.708 -> Kelas: Dingin  
6. Jarak: 8.062 -> Kelas: Dingin  
7. Jarak: 8.544 -> Kelas: Panas  
8. Jarak: 9.487 -> Kelas: Panas
```

Gambar 1. 5

Pada *Gambar 1.5* di atas, merupakan sebuah code untuk melakukan perhitungan jarak euclidean.

```

print("\n=== Prediksi KNN Menggunakan Scikit-Learn ===")
for k in Ks:
    # Membuat model KNN dengan jumlah tetangga K
    knn = KNeighborsClassifier(n_neighbors=k)

    # Training model menggunakan data training
    knn.fit(X, y)

    # Prediksi kelas untuk data test
    prediction = knn.predict(x_test)[0]

    print(f"K = {k} -> Prediksi: {'Panas' if prediction else 'Dingin'}")

=== Prediksi KNN Menggunakan Scikit-Learn ===
K = 1 -> Prediksi: Dingin
K = 3 -> Prediksi: Dingin
K = 5 -> Prediksi: Dingin
K = 7 -> Prediksi: Dingin

```

Gambar 1. 6

Pada *Gambar 1.6* di atas, merupakan sebuah code untuk memprediksi dengan menggunakan model KNN dengan jumlah tetangga K yaitu 1,3,5,7. Dengan artian kecepatan angin 3KM dan temperatur suhu adalah 16 derajat termasuk kedalam kategori 'dingin'.

2. Tugas mandiri – Latihan2

```
df_2 = pd.read_excel(path + '/praktikum_mandiri/data/latihan2.xlsx')
df_2
```

	NIM	Hasil Sebenarnya	Hasil Prediksi
0	TI001	Lulus	Lulus
1	TI002	Lulus	Lulus
2	TI003	Lulus	Lulus
3	TI004	Lulus	Tidak Lulus
4	TI005	Lulus	Tidak Lulus
5	TI006	Tidak Lulus	Lulus
6	TI007	Tidak Lulus	Tidak Lulus
7	TI008	Tidak Lulus	Tidak Lulus
8	TI009	Tidak Lulus	Tidak Lulus
9	TI010	Tidak Lulus	Tidak Lulus

Gambar 2.1

Pada *Gambar 2.1* di atas, merupakan sebuah code untuk menampilkan dataset menggunakan tabel dengan menggunakan dataset latihan2. Memprediksi kelulusan matakuliah.

```
y_true = df_2['Hasil Sebenarnya']
y_pred = df_2['Hasil Prediksi']
```

Gambar 2.2

Pada *Gambar 2.2* di atas, merupakan sebuah code untuk mengambil kolom data aktual dan prediksi.

```
cm = confusion_matrix(y_true, y_pred, labels=['Lulus', 'Tidak Lulus'])
```

Gambar 2.3

Pada *Gambar 2.3* di atas, merupakan sebuah code untuk membuat confusion matrix dengan label Lulus dan Tidak Lulus.

```
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, pos_label='Lulus')
recall = recall_score(y_true, y_pred, pos_label='Lulus')
f1 = f1_score(y_true, y_pred, pos_label='Lulus')

print("=== Evaluasi Model Prediksi Kelulusan ===")
print("Confusion Matrix:")
print(cm)
print("\nAccuracy:", round(accuracy, 2))
print("Precision:", round(precision, 2))
print("Recall:", round(recall, 2))
print("F1-Score:", round(f1, 2))

=== Evaluasi Model Prediksi Kelulusan ===
Confusion Matrix:
[[3 2]
 [1 4]]

Accuracy: 0.7
Precision: 0.75
Recall: 0.6
F1-Score: 0.67
```

Gambar 2.4

Pada *Gambar 2.4* di atas, merupakan sebuah code untuk menampilkan hasil dari accuracy, precision, recall dan f1-Score pada dataset yang telah dimiliki.

2. Tugas mandiri – Latihan2

```
df_3 = pd.read_excel(path + '/praktikum_mandiri/data/latihan3.xlsx')
df_3
```

	Temperature	Humidity	Wind Speed	Precipitation(%)	Cloud Cover	Atmospheric Pressure	UV Index
0	14.0	73	9.5	82.0	partly cloudy	1010.82	2
1	39.0	96	8.5	71.0	partly cloudy	1011.43	7
2	30.0	64	7.0	16.0	clear	1018.72	5
3	38.0	83	1.5	82.0	clear	1026.25	7
4	27.0	74	17.0	66.0	overcast	990.67	1
5	32.0	55	3.5	26.0	overcast	1010.03	2
6	-2.0	97	8.0	86.0	overcast	990.87	1
7	3.0	85	6.0	96.0	partly cloudy	984.46	1
8	3.0	83	6.0	66.0	overcast	999.44	0
9	28.0	74	8.5	107.0	clear	1012.13	8

Gambar 3.1

Pada *Gambar 3.1* di atas, code untuk menampilkan dataset yang digunakan untuk latihan3. Pada sebuah tabel menggunakan dataset untuk prediksi cuaca.

```
X = df_3[['Temperature', 'Humidity', 'Wind Speed', 'Precipitation(%)', 'Atmospheric Pressure', 'UV Index']]
y_test= df_3['Cloud Cover']
```

Gambar 3.2

Pada *Gambar 3.1* di atas, code untuk menentukan fitur X dan Y.

```
encoder = LabelEncoder()
y = encoder.fit_transform(y_test)
```

Gambar 3.3

Pada *Gambar 3.3* di atas, code untuk melakukan encode label pada fitur Y yaitu Cloud Cover.

```
print("Pemetaan Label Target:")
for i, label in enumerate(encoder.classes_):
    print(f"{label}: {i}")

Pemetaan Label Target:
clear: 0
overcast: 1
partly cloudy: 2
```

Gambar 3.4

Pada *Gambar 3.4* di atas, merupakan code untuk menampilkan hasil encode label pada fitur Y yaitu Cloud Cover.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)
```

Gambar 3.5

Pada *Gambar 3.5* di atas, merupakan code untuk membagi data training dan data testing untuk melakukan pengujian.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nBentuk data latih (X_train):", X_train_scaled.shape)
print("Bentuk data uji (X_test):", X_test_scaled.shape)

Bentuk data latih (X_train): (7, 6)
Bentuk data uji (X_test): (3, 6)
```

Gambar 3.6

Pada *Gambar 3.6* di atas, merupakan code untuk scaling fitur menggunakan *StandardScaler* yang akan digunakan untuk model KNN.


```

k = 5
knn_model = KNeighborsClassifier(n_neighbors=k)

knn_model.fit(X_train_scaled, y_train)

print(f"\nModel KNN dengan K={k} telah dilatih.")

```

Gambar 3.7

Pada *Gambar 3.7* di atas, merupakan sebuah code untuk melatih model menggunakan KNN dengan menginisiasi K-tetangga adalah 5 dan menggunakan data yang telah di-scale

```

y_pred = knn_model.predict(X_test_scaled)

accuracy = accuracy_score(y_test, y_pred)
print(f"\n--- Hasil Evaluasi Model (K={k}) ---")
print(f"Akurasi Model: {accuracy*100:.2f}%")

# 3. Tampilkan Laporan Klasifikasi
print("\nLaporan Klasifikasi:")
# Target names adalah label asli (partly cloudy, clear, overcast)
target_names = encoder.classes_
print(classification_report(y_test, y_pred, target_names=target_names))

```

```

--- Hasil Evaluasi Model (K=5) ---
Akurasi Model: 66.67%

```

Laporan Klasifikasi:

	precision	recall	f1-score	support
clear	1.00	1.00	1.00	1
overcast	0.50	1.00	0.67	1
partly cloudy	0.00	0.00	0.00	1
accuracy			0.67	3
macro avg	0.50	0.67	0.56	3
weighted avg	0.50	0.67	0.56	3

Gambar 3.7

Pada *Gambar 3.7* di atas, merupakan sebuah code untuk menampilkan hasil dari pengujian model menggunakan KNN dengan menampilkan `accuracy_score` dan `classification_report`. Dengan hasil akurasi yaitu sebesar 66.67%.

Link github praktikum:

https://github.com/MuhZaidanRamdhan/machine-learning/blob/main/pertemuan10/praktikum_10/notebooks/praktikum_10.ipynb

Link github tugas:

https://github.com/MuhZaidanRamdhan/machine-learning/blob/main/pertemuan10/praktikum_mandiri/notebooks/latihan_10.ipynb